CAS CS 538. Solutions to Problem Set 11

Due electronically via Gradescope, Monday April 27, 2026 11:59pm

Problem 1. (60 points) Let (G, S, V) be a secure signature scheme. Consider the following signature scheme:

- $G'()$: same as G. Return (pk, sk).
- $S'(sk, m)$: run G to sample a temporary key pair (pk_t, sk_t). Produce $\tau = S(sk, pk_t)$ (τ signs the temporary public key with the permanent key) and $\rho = S(sk_t, m)$ (ρ signs the message with the temporary signing key). Output $\sigma' = (pk_t, \tau, \rho)$.
- $V'(pk, m, (pk_t, \tau, \rho))$: Output $V(pk, pk_t, \tau) \wedge V(pk_t, m, \rho)$.

Note that this way of doing signatures is similar to what happens in certificate chains (although the same pk_t may be used multiple times). It also happens in signature delegation: I can send sk_t along with τ to someone else when I want to delegate the power to sign with my key temporarily to them (for example, if I am absent). The temporary key can then be publicly revoked. This approach could also be used for delegation to multiple parties, followed by traitor tracing: if sk_t is used by a traitor to sign a message that should not be signed, then the traitor's signature will contain pk_t and thus reveal who the traitor is.

Prove that (G', S', V') is secure via the following two steps. Consider an adversary $\mathcal{A}$ who makes q chosen-message attack queries and then forges a valid signature $\sigma' = (pk_t, \tau, \rho)$ for a new message with nonnegligible probability p .

(a) (30 points) Let

- X be the event that $\mathcal{A}$ succeeds and
- Y be the event that the value pk_t in σ' was at some point given to $\mathcal{A}$ as part of the response to any of its chosen-message attack queries.

Show a reduction that breaks S with probability polynomially related to $\Pr[X \wedge Y]$.

Solution. Let $\mathcal{B}$ be an efficient adv, that uses $\mathcal{A}$ to break the EUF-CMA security of (G, S, V).

$\mathcal{B}$ is given a public key, pk, and oracle access to $S(sk, \cdot)$. It will guess a rnd index $i^* \xleftarrow{R} \{1, \ldots, q\}$ and runs $\mathcal{A}$ as follows:

On the i-th signing query m_i from $\mathcal{A}$:

- **IF** $i = i^*$
  1. Set $pk_t := pk$.
  2. Query the signing oracle, to get $\tau \leftarrow S(sk, pk)$.
  3. Query the signing oracle again, to get $\rho \leftarrow S(sk, m_{i^*})$.
  4. Return $\sigma_{i^*} = (pk_t, \tau, \rho)$ to $\mathcal{A}$.

- **ELSE** ($i \neq i^*$)
  1. Generate $(pk_t, sk_t) \leftarrow G()$.
  2. Query the signing oracle, to get $\tau \leftarrow S(sk, pk_t)$.
  3. Calculate $\rho \leftarrow S(sk_t, m_i)$.
  4. Return $\sigma_i = (pk_t, \tau, \rho)$ to $\mathcal{A}$.

Correctness of Simulator For $i \neq i^*$, this is a perfect simulation of S'. For $i = i^*$, we will have $pk_t = pk$, $\tau = S(sk, pk)$, and $\rho = S(sk, m_{i^*})$, so:

$$V'(pk,\ m_{i^*},\ (pk, \tau, \rho)) = V(pk, pk, \tau) \wedge V(pk, m_{i^*}, \rho) = 1,$$

which is also a valid response. Therefore, the simulation is perfect in both cases.

Forgery Eventually, $\mathcal{A}$ outputs (m^*, σ^*) where $\sigma^* = (pk_t^*, \tau^*, \rho^*)$. $\mathcal{B}$ will output the pair (m^*, ρ^*) as its forgery.

Correctness Now suppose that $X \wedge Y$ holds and $\mathcal{B}$'s guess satisfies $pk_t^* = pk$ (basically that the response to query i^* was the one whose pk_t appeared in $\mathcal{A}$'s forgery). Then:

- $V'(pk, m^*, (pk, \tau^*, \rho^*)) = 1$ implies that $V(pk, m^*, \rho^*) = 1$.
- Since (m^*, σ^*) is a valid forgery for $\mathcal{A}$, we will then have $m^* \notin \{m_1, \ldots, m_q\}$.
- $\mathcal{B}$'s signing oracle was queried on msgs $\{pk,\ m_{i^*}\} \cup \{pk_{t,j}\}_{j \neq i^*}$, but never on m^* (since $m^* \neq m_{i^*}$, as m^* must be fresh).

Therefore (m^*, ρ^*) is a valid EUF-CMA forgery against (G, S, V).

Success % $\mathcal{B}$ will succeed whenever $X \wedge Y$ occurs and $\mathcal{B}$ correctly guessed i^*, which happens with probability $1/q$. Therefore:

$$\Pr[\mathcal{B} \text{ succeeds}] \ \geq\ \frac{1}{q} \cdot \Pr[X \wedge Y].$$

And if $\Pr[X \wedge Y]$ is non-negligible, then $\mathcal{B}$ will then break the EUF-CMA security of (G, S, V) with non-negligible probability as well.

(b) (30 points) Use part (a) and some additional work to prove that (G', S', V') is secure. Hint: $p = \Pr[X] = \Pr[X \wedge Y] + \Pr[X \wedge \neg Y]$. Part (a) helps when the first of these addends is at least $p/2$. Now think what to do when it's $p/2$ or less. You'll need a second reduction.

Solution. By combining the reduction from (1a) with another reduction, handling the complementary case, we can prove that (G', S', V') is secure.

Setup Let $\mathcal{A}$ be an efficient adv, that can forge a valid signature under (G', S', V') with non-negligible probability $p = \Pr[X]$. By the hint, we know that at least one term has gotta be $\geq p/2$. Each case

can be handled with their own reduction:

The case that $\Pr[X \wedge Y] \geq p/2$**:** Let $\mathcal{B}_1$ just be the reduction from (1a). From (1a), we already know that $\mathcal{B}_1$ breaks (G, S, V) with probability:

$$\Pr[\mathcal{B}_1 \text{ succeeds}] \;\geq\; \frac{1}{q} \cdot \Pr[X \wedge Y] \;\geq\; \frac{p}{2q},$$

which is non-negligible, since p is non-negligible and q is polynomial.

The case that $\Pr[X \wedge \neg Y] \geq p/2$**:** Let $\mathcal{B}_2$ be a new adv, given pk and oracle access to $S(sk, \cdot)$. It will run $\mathcal{A}$ and simulates the signing oracle as follows:

On the i-th signing query m_i from $\mathcal{A}$:

1. Generate $(pk_t, sk_t) \leftarrow G()$.
2. Query the signing oracle, to get $\tau \leftarrow S(sk, pk_t)$.
3. Calculate $\rho \leftarrow S(sk_t, m_i)$.
4. Return $\sigma_i = (pk_t, \tau, \rho)$ to $\mathcal{A}$.

Note how this is the same perfect simulation of S' used in (1a) (applied to every query).

Forgery When $\mathcal{A}$ outputs (m^*, σ^*) where $\sigma^* = (pk_t^*, \tau^*, \rho^*)$, $\mathcal{B}_2$ will output (pk_t^*, τ^*) as its EUF-CMA forgery.

Correctness If $X \wedge \neg Y$ occurs, then:

- $V'(pk, m^*, (pk_t^*, \tau^*, \rho^*)) = 1$, which implies $V(pk, pk_t^*, \tau^*) = 1$.
- $\neg Y$ means that pk_t^* never showed up as pk_t in any oracle response. And Since $\mathcal{B}_2$'s only signing oracle queries are $\{pk_{t,1}, \ldots, pk_{t,q}\}$, and $pk_t^* \notin \{pk_{t,1}, \ldots, pk_{t,q}\}$, the msg pk_t^* had to have been *never queried* by $\mathcal{B}_2$.

Therefore (pk_t^*, τ^*) is a valid EUF-CMA forgery against (G, S, V), and:

$$\Pr[\mathcal{B}_2 \text{ succeeds}] = \Pr[X \wedge \neg Y] \;\geq\; \frac{p}{2}.$$

End Both cases managed to break the EUF-CMA security of (G, S, V) with non-negligible probability. This contradicts the initial assumed security, proving that (G', S', V') is secure. $\square$

Problem 2. (40 points) In the Schnorr identification protocol (section 19.1), the prover is supposed to sample a fresh uniformly random value $\alpha_t \xleftarrow{\text{R}} \mathbb{Z}_q$. But what if the prover does not correctly sample α_t — for example, what if the α_t values for different protocols are not independent, even though each

one is individually random? Such an implementation mistake could doom security, as demonstrated by attacks on Sony Playstation 3 and some Bitcoin wallet implementations (see the end of 19.3 in the textbook). The attacks happened in signature schemes rather than identification protocols, but the idea is the same. It is actually challenging to get fresh randomness for every signature generated, which is why a preferred implementation these days is to generate α_t as a PRF applied to the message being signed; that way, fresh randomness needs to be generated only once for the PRF seed, at the same time as (pk, sk) are generated). (See Excercise 13.6 if you are interested in the details.)

Specifically for this problem, consider a prover who runs the Schnorr identification protocol with a verifier twice. In the first interaction, the prover uniformly samples $\alpha_{t,1} \xleftarrow{\text{R}} \mathbb{Z}_q$. In the second, they set $\alpha_{t,2} \equiv_q a \cdot \alpha_{t,1} + b$, for some known $a, b \in \mathbb{Z}_q$. The prover uses the same secret α in both interactions. Suppose the verifier knows a and b, their challenge values c_1 and c_2, and learns the prover's messages $(u_{t,i}, \alpha_{z,i})$ for $i \in \{1, 2\}$. Show how the verifier can learn, with high probability, the prover's secret α.

Solution. Looking back at Schnorr's ID protocol, note how each accepting conversation satisfies the following:

$$\alpha_{z,i} \equiv_q \alpha_{t,i} + \alpha \cdot c_i.$$

The 2 interactions, by the question, gives the verifier the following system:

$$\alpha_{z,1} \equiv_q \alpha_{t,1} + \alpha \cdot c_1, \tag{1}$$
$$\alpha_{z,2} \equiv_q \alpha_{t,2} + \alpha \cdot c_2 \equiv_q a \cdot \alpha_{t,1} + b + \alpha \cdot c_2. \tag{2}$$

Now from the 1st equation, we can isolate $\alpha_{t,1}$:

$$\alpha_{t,1} \equiv_q \alpha_{z,1} - \alpha \cdot c_1.$$

And then sub that into the second equation:

$$\alpha_{z,2} \equiv_q a(\alpha_{z,1} - \alpha \cdot c_1) + b + \alpha \cdot c_2 = a \cdot \alpha_{z,1} + b + \alpha(c_2 - a \cdot c_1).$$

And then just a bit of rearranging gets:

$$\alpha \cdot (c_2 - a \cdot c_1) \equiv_q \alpha_{z,2} - a \cdot \alpha_{z,1} - b.$$

Now since q is prime, if $c_2 - a \cdot c_1 \not\equiv_q 0$, then the verifier will recover:

$$\alpha \equiv_q \big(\alpha_{z,2} - a \cdot \alpha_{z,1} - b\big) \cdot \big(c_2 - a \cdot c_1\big)^{-1} \pmod{q}.$$

Probability of getting α Note that the denominator is zero only when $c_2 \equiv_q a \cdot c_1$. Now, since c_1 is fixed and c_2 is drawn uniformly at rnd from C (which is super-poly by assumption [Boneh-Shoup 19.1]), this event will occur with probability AT MOST $1/|C|$, which is negligible. Therefore, the verifier should be able to recover α with probability $\geq 1 - 1/|C|$.

Note How this extraction works is pretty similar to the rewinding technique (Boneh-shoup 19.1). It basically shows how two accepting conversations, sharing the same u_t but differing in challenges, is enough to compute the discrete log.

Basically, having an expression of this structure specfically:

$$\alpha_{t,2} = a \cdot \alpha_{t,1} + b$$

is enough to get rid of the unknown $\alpha_{t,1}$ algebraically, replacing the rewind with a direct sub.